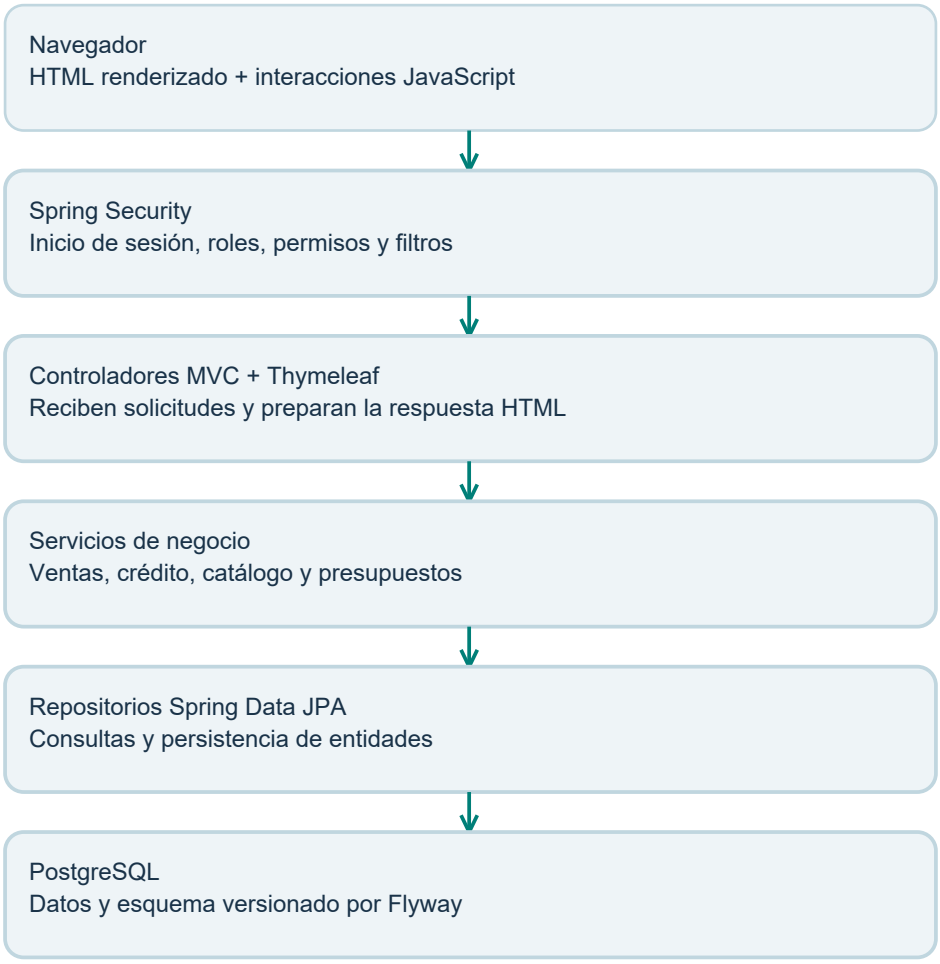
Cómo está construido

Monolito web MVC organizado por capas técnicas y dominios.

Maidav reúne interfaz, negocio y acceso a datos en una aplicación Java desplegable como una unidad. PostgreSQL y los archivos subidos conservan el estado persistente.



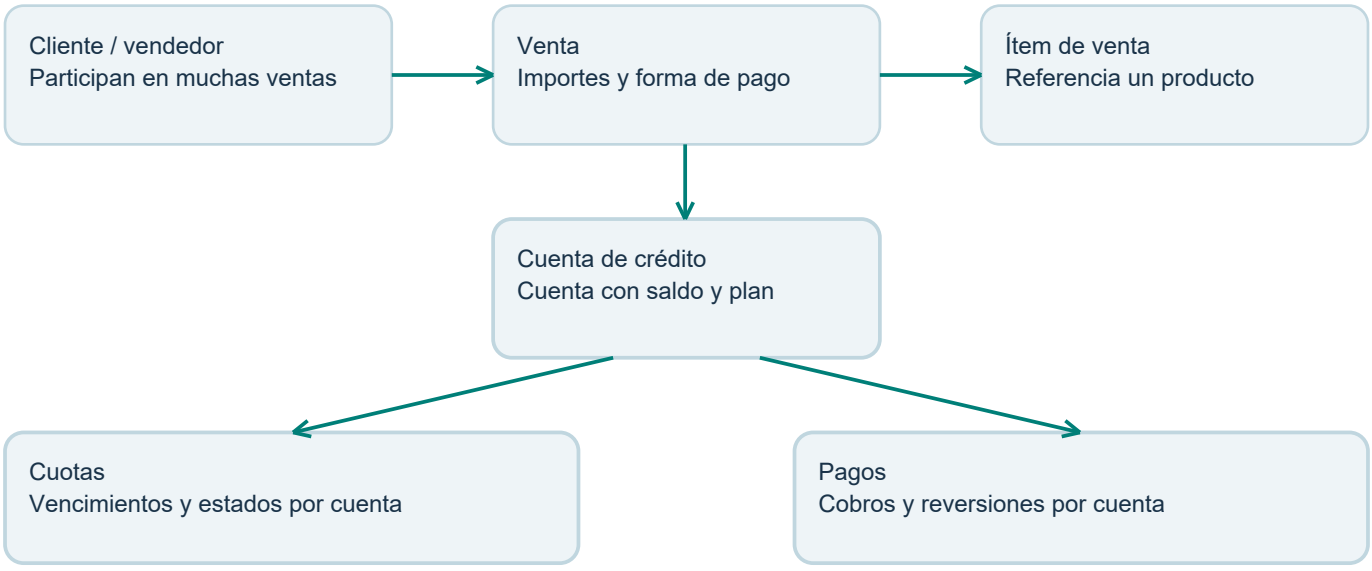
Componente	Tecnología observada
Backend	Java 21; Spring Boot 3.5.9; Spring MVC, Security y Data JPA.
Interfaz	Thymeleaf, recursos estáticos, Tailwind y Alpine.js por CDN; endpoints JSON puntuales.
Datos / ejecución	PostgreSQL 16 en Compose; 53 migraciones Flyway (V1-V53); Maven y Docker.

Módulos y relaciones

Todos comparten aplicación y base de datos.

Módulo	Responsabilidad	Ejemplos de código
Identidad	Usuarios, roles, permisos y acceso.	User, Role, Permission; UserServiceImpl.
Clientes	Clientes y zonas.	Client, Zone; ClientServiceImpl.
Catálogo	Productos, stock, proveedores, precios e imágenes.	ProductServiceImpl; ProductRepository.
Ventas y crédito	Ventas, cuotas, cobros, mora y anulaciones.	SaleServiceImpl; CreditAccountServiceImpl.
Presupuestos	Planes, cálculo y documentos PDF.	QuoteCalculator; QuoteServiceImpl.
Administración	Dashboard, empresa y exportaciones.	DashboardPortfolioService; ExportDocumentService.

Relaciones centrales, simplificadas



La venta financiada crea una cuenta y su calendario de cuotas. Los pagos se relacionan con la cuenta; el dibujo no representa una relación única directa entre pago y cuota.

Carpetas: web/controller/web = HTTP; service/<dominio> = negocio; repository/<dominio> = datos; model/<dominio> = entidades; resources/templates = pantallas; db/migration = esquema.

Patrones que utiliza

Significado breve y evidencia concreta en el proyecto.

Patrón	Qué hace y dónde aparece
MVC + capas	SaleController recibe HTTP; Thymeleaf genera HTML; SaleServiceImpl coordina negocio. Algunos controladores también acceden a repositorios: la separación no es estricta.
Service Layer	Servicios coordinan reglas. Es común la interfaz *Service con *ServiceImpl; también hay servicios y clases de apoyo concretas.
Repository + ORM	JpaRepository encapsula consultas; @Entity mapea objetos a tablas. Hay JPQL y SQL nativo: existe acoplamiento con JPA/PostgreSQL.
Inyección de dependencias	Spring conecta componentes mediante constructores; Lombok @RequiredArgsConstructor genera varios de ellos.
Transacciones	@Transactional agrupa operaciones. Crear una venta incluye stock, cabecera, ítems y, si corresponde, cuenta y cuotas.
Post/Redirect/Get	SaleController redirige después de guardar, reduciendo reenvíos al recargar una pantalla.
Idempotencia	Crear una venta desde MVC usa UUID por intento y usuario. V52 define unicidad; el repositorio usa PESSIMISTIC_WRITE. Repetir la misma clave recupera la venta creada.
RBAC + permisos	Spring Security y @PreAuthorize controlan acciones. AuthenticationRefreshFilter refresca autoridades por intervalo: PT5M por defecto.
Auditoría y migraciones	BaseEntity registra creación/modificación con JPA Auditing. Flyway versiona el esquema. Esto no equivale a un historial completo de cada cambio.

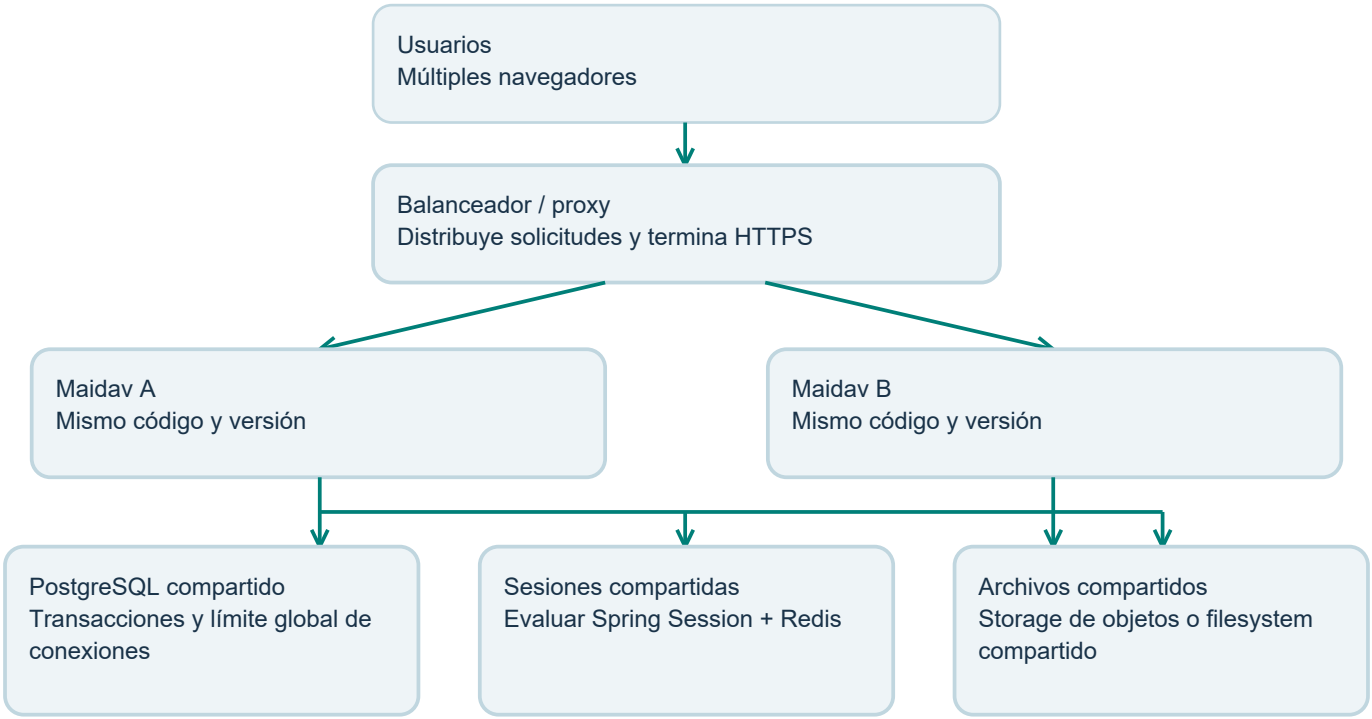
Qué arquitectura describe esto

Es un monolito por capas con agrupación por dominio. Los dominios aún no son módulos aislados. No se observan puertos/adaptadores de arquitectura hexagonal ni servicios desplegables de forma independiente.

Diseño para escalar

Propuesta de evolución: los componentes nuevos aún no están implementados.

Mantener inicialmente el monolito y preparar varias instancias. Replicar la aplicación exige resolver sesiones y archivos compartidos, además de medir PostgreSQL.



Dependencia actual	Cambio necesario para replicar
Sesión HTTP	Hay HttpSession y sincronización local en el filtro. Validar serialización y concurrencia con sesiones compartidas. La afinidad sola no resuelve caídas.
Imágenes en disco	/uploads/** es público y depende del filesystem. Compartir almacenamiento, conservar URLs y definir acceso por tipo de archivo.
Concurrencia comercial	El stock ya usa decremento condicional atómico. Conservarlo; probar cobros y anulaciones simultáneos. La idempotencia de ventas no cubre todos los flujos.
Base de datos	Las réplicas multiplican conexiones. Medir consultas, bloqueos y pool total antes de sumar instancias.

Ruta de crecimiento

Orden recomendado para evolucionar sin perder consistencia comercial.

Paso	Trabajo propuesto	Señal para avanzar
1. Medir y proteger	Medir latencia p95, errores, recursos y consultas. Automatizar pruebas críticas y verificar restauración de datos e imágenes.	Carga de referencia y pruebas de ventas, cobros, stock, permisos y reintentos.
2. Optimizar	Revisar consultas lentas, cargas masivas y paginación. V53 ya agrega índices a ventas. Ajustar recursos según mediciones.	Meta de latencia y errores acordada, cumplida bajo carga representativa.
3. Ordenar módulos	Delimitar catálogo, ventas, cobranza e identidad; extraer responsabilidades de clases grandes y mantener transacciones coherentes.	Cambiar un módulo con pruebas sin depender de detalles internos de otro.
4. Replicar	Compartir sesiones y archivos; balanceador, observabilidad y despliegue controlado; coordinar migraciones Flyway.	Caída de una instancia sin pérdida de archivos ni duplicación de operaciones.
5. Aislar cargas	Si las métricas lo justifican, pasar exportaciones pesadas a trabajos en segundo plano con reintentos e idempotencia.	Menor demora de pantallas y trazabilidad de cada trabajo.

Microservicios: evaluarlos cuando un módulo necesite escalar o desplegarse de forma independiente y el equipo pueda operarlo. Separar ventas, stock y crédito agrega coordinación entre servicios; se recomienda mantenerlos juntos al principio.

Fuentes y alcance

Código: pom.xml; src/main/java/com/sales/maidav/ (SaleController, SaleServiceImpl, ProductRepository, SaleIdempotencyKeyRepository, SecurityConfig, AuthenticationRefreshFilter, BaseEntity y entidades); src/main/resources/db/migration/; Dockerfile; docker-compose.yml; render.yaml.

Contexto: specs/current-system.md; adr/ADR-000-current-architecture.md; tasks/technical-debt.md. El código revisado actualiza el relevamiento previo: V1-V53, 15 archivos de prueba Java y una prueba de navegador; no se ejecutaron.

El Dockerfile construye omitiendo pruebas. No se detectó CI versionada ni almacenamiento distribuido de sesiones en las dependencias revisadas. Render y Compose describen despliegues posibles; no certifican el entorno productivo vigente.

Relevamiento estático: no se midió capacidad máxima ni rendimiento. La evolución es una propuesta; no se modificaron código ni infraestructura de la aplicación.